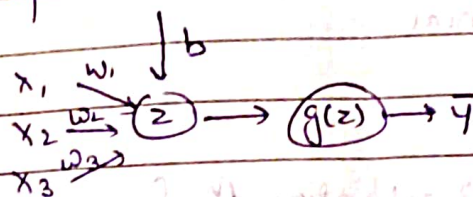


Neural Network

Date: / / Page no:

Similar to logistic Regression (perceptron) but with more layers & hidden units.



$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$g(z)$ is Activation fⁿ

→ Sigmoid $\sigma(z) = \frac{1}{1 + e^{-z}}$

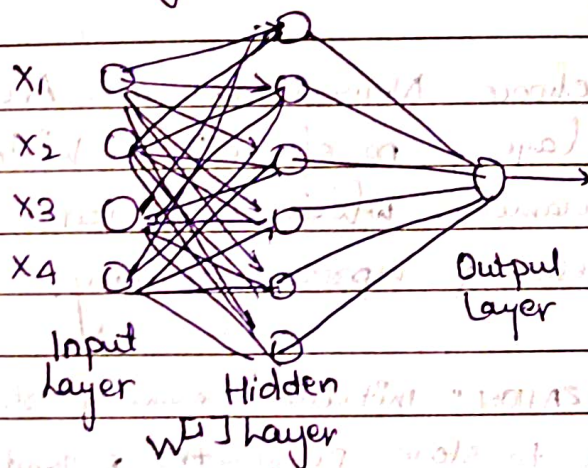
→ tanh $\sigma(z) = \frac{e^{-z} - e^z}{e^{-z} + e^z}$

→ relu $\sigma(z) = \max(0, z)$

→ leaky relu

FOR CLASSIFICATION → Sigmoid

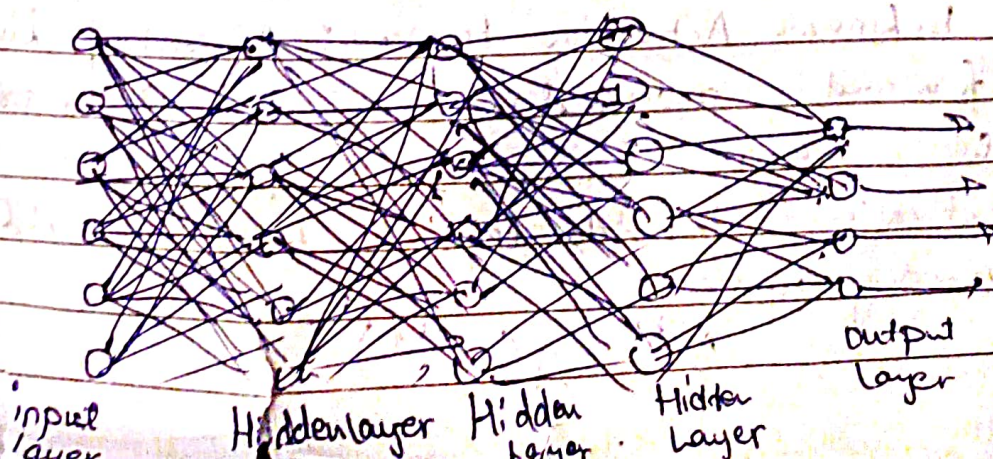
SIMPLE one layer Neural Network



→ have weighted sum of i/p's with parameter (w) followed by activation fⁿ
there will be bias (b) for each hidden layer

for 2 class classifier Problem, output layer will be probability that given training example belong to the class (ci)

4 layered Deep Neural Network



For Single layered :

$$z = W^T X + b$$

↓ weighted sum of i/p, b is bias

$$g(z)$$

In Practice we choose RELU activation fⁿ for hidden layer & sigmoid fⁿ for o/p layer.

B/c using sigmoid will lead to vanishing gradient.

STEPS

1. ARCHITECTURE : choose Neural Network Architecture i.e. no. of hidden layer, no. of units in hidden layers. These are hyperparameters which are needed to be tuned for better result & improved accuracy.

2. PARAMETER INITIALIZATION : initialize parameter randomly. we use dictionaries to store parameters & gradients i.e. $W_i = ["w" + \text{str}(i)]$ & $dw_i = \text{grads}["dw" + \text{str}(i)]$

3. Forward Propagation - Once weights are initiated, input X is propagated through hidden ~~to~~ units at each layer & produce the output $\bar{y}$

a. Linear forward : At any layer i

$$z^{[i]} = W^{[i]} A^{[i-1]} + b^{[i]}$$

when $A^{[0]} = X$

b. Linear Activation forward : Result of Linear forward is processed into activation fⁿ to produce the o/p layer.

Activation fⁿ in hidden layer is generally RELU to avoid vanishing gradient.

$$A^{[i]} = g(z^{[i]})$$

4. Compute Cost : Cost fn used in logistic regression
Cross Entropy Cost Once we have final layer's O/P A' ,

$$\text{Cost} = \left(\frac{-1}{m} \right) \left(y * \log(A') + (1-y) * \log(1-A') \right)$$

y is true O/P

A' is O/P of network

m is no. of training examples

5. Backward Propagation :

it gives gradients of parameters & bias which are used in gradient descent step to update weights.

a. Linear backward Suppose you have already calculated derivative dz^l we want to get $dw^l, db^l, dA^{[l-1]}$.

We use 3 formula :-

$$dw^{[l]} = \frac{1}{m} dz^{[l]} A^{[l-1]T}$$

$$db^{[l]} = \frac{1}{m} \sum_{i=1}^m dz^{[l]}(i)$$

$$dA^{[l-1]} = W^{[l]T} dz^{[l]}$$

b. Linear Activation backward

to compute dz^l ,

$$dz^{[l]} = dA^{[l]} * g'(z^{[l]})$$

$\hookrightarrow$ derivative of $g(z)$

it can be relu or sigmoid based

if dz^l is final o/p layer

$g(z)$ is Sigmoid for other $g(z)$ is relu

6. Update Weights :-

finally update weights which we get during b/w propagation.

We use Gradient descent to update weights.

$$W := W - \alpha \cdot \frac{\partial J}{\partial W}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

$\alpha \rightarrow$ learning rate is hyperparameter

TIPS :

- Try different values for learning rate, No of layers, No of hidden units in each layer etc.
- In this we use 3 hidden layers with 60, 10, 10 units in each hidden layers.
- Always monitor cost f^n . If cost f^n is not decreasing with iterations, then there is surely a problem with your model.

Hyper Parameter

Regularization Parameter (λ)

(i) Learning Rate (α)

No of Units in each hidden layer

No of hidden layers

Perceptron

is one of the simplest artificial neural network architectures, introduced by Frank Rosenblatt in 1957. It is primarily used for binary classification.

Traditionally Statistical ML & Conventional Programming were used for predictions.

The Perceptron model proved to be highly effective in solving specific classification problems, laying the ground work for advancement in AI & ML.

- It is type of neural network
- Performs binary classification that maps input features to an output decision, usually classification data into one of two categories such as 0 or 1.
- Single layer of input nodes that are fully connected to layer of output nodes.
- Good at learning linearly separable patterns.
- Utilize variation of artificial neurons called Threshold Logic Units (TLU)

Types of Perceptron

1. Single layer
2. Multi-layer → consist two or more layer, more complex patterns

BASIC COMPONENTS OF PERCEPTRON

Input Features : ^{Per}takes multiple input features, each representing a characteristic of the input data.

Weights : Each input feature is assigned a weight that determines its influence on the output. adjusted during training to find optimal values

Summation f^n perceptron calculates the weighted sum of its i/p's, combining them with their respective weights.

Activation f^n - Weighted sum is passed through a heaviside step f^n , comparing it to threshold to produce binary o/p (0 or 1).

Output: final o/p is determined by activation f^n , often used for binary classification tasks.

Bias: bias term helps the perceptron make adjustments independent of i/p, improving flexibility in learning.

Learning Algorithm: Perceptron adjusts its weights & bias using a learning algorithm, such as Perceptron Learning Rule, to minimize prediction errors.

Perceptron Learning Algorithm

4 significant steps :-

1. First multiply all i/p values with corresponding weight values & then add them to determine the weighted sum.

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots + w_n * x_n$$

Add another essential term called bias 'b' to weighted sum to improve the model performance.

$$\sum w_i * x_i + b$$

2. Next an activation f^n is applied to this weighted sum, producing binary or continuous-valued output.

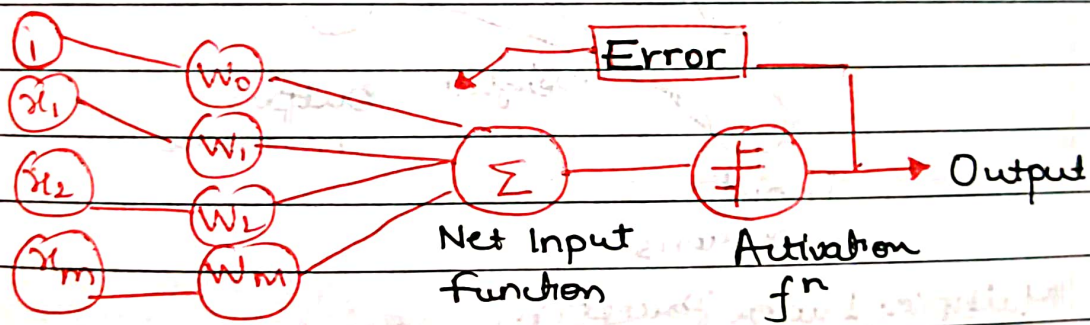
$$Y = f(\sum w_i * x_i + b)$$

Next the difference b/w this o/p & actual target value is computed get the error term, E , generally in terms of mean squared error. The steps up to this form the forward propagation part of the algorithm.

$$E = (Y - Y_{\text{actual}})^2$$

We optimize this error (loss f^n) using an optimization algorithm. Generally some of gradient descent algorithm is used to find optimal values of hyperparameters like learning rate, weight & Bias.

This step forms the backward propagation part of Alg.

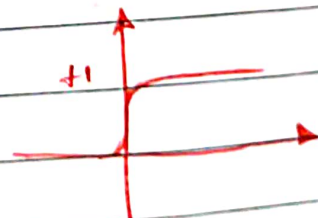
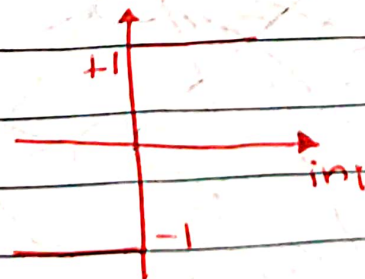
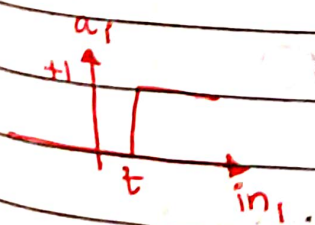


Perceptron Rule

Step f^n

Sign f^n

Sigmoid f^n

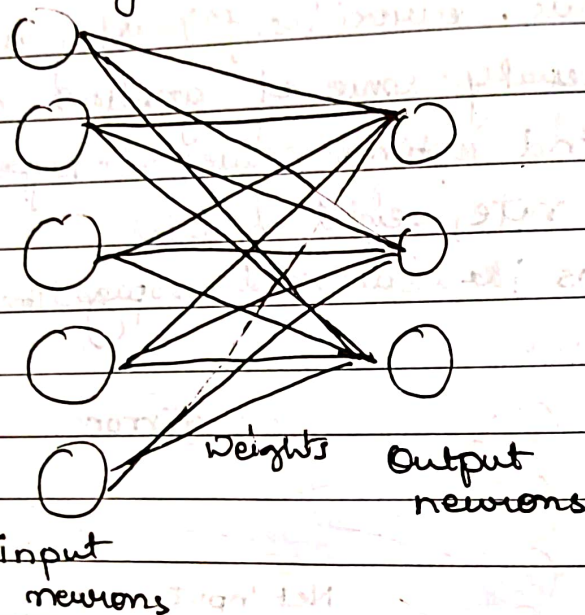


Single Layer Perceptron Model

→ Simplest ANN model.

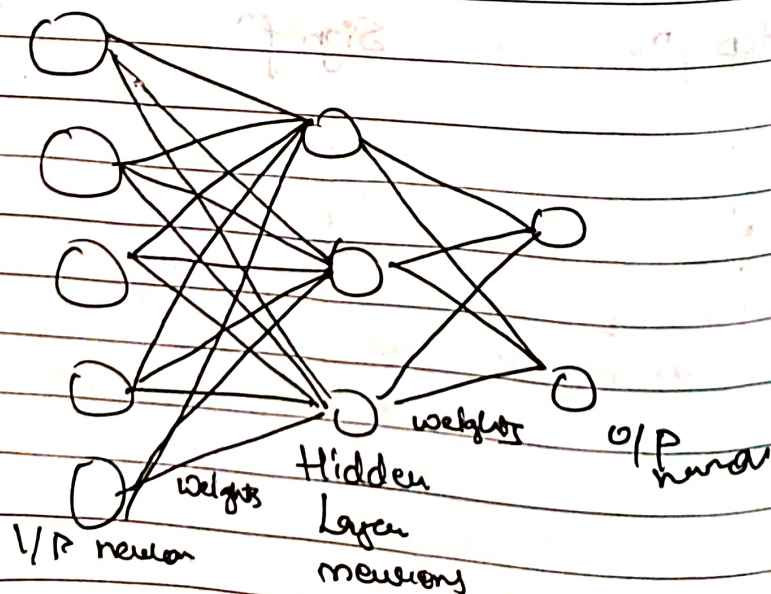
→ Consist of a feed forward network & includes a threshold transfer f for thresholding on the O/P.

→ Main objective is to classify linearly separable data with binary labels.



Multiple Layer Perceptron

→ Same structure but consists of additional one or more hidden layers.



GRADIENT DESCENT



Page No: _____

Date: ____/____/____

backbone of learning process of various algorithms like Linear Regression, logistic regression, SVM & neural network.

It serves as fundamental optimization technique to minimize cost fn of a model.

→ Iteratively adjusting the model parameters to reduce the difference b/w predicted & actual values.

Backpropagation computes the gradients of loss fn wrt each parameter (weight & biases) in the network by applying chain rule.

1. Linear Regression

Gradient descent minimizes the MSE, MSE is convex & gradient descent guarantee convergence to the global minimum if learning rate is appropriately chosen.

$$w = w - \alpha \cdot \frac{\partial J(w, b)}{\partial w}, \quad b = b - \alpha \cdot \frac{\partial J(w, b)}{\partial b}$$

2. Logistic Regression

Minimize LogLoss (Cross Entropy loss) to optimize decision boundary for binary classification. ∵ op is probabilities sigmoid fn is applied.

$$w = w - \alpha \frac{\partial J(w)}{\partial w}$$

3. Support Vector Machine

Optimize the hinge loss which ensures a maximum margin hyperplane.
(Q) It ensures the optimal placement of hyperplane to separate classes with largest possible margin.

VANISHING & EXPLODING GRADIENT Problems.

Vanishing Gradient emerges during backpropagation when derivatives or slopes of the activation f' become progressively smaller as we move backward through the layers of a neural network.

The weight updates becomes extremely tiny, or even exponentially small, it can significantly prolong the training time & worst case it can halt the training processing.

It is particularly associated with Sigmoid & hyperbolic tangent (tanh) Activation f' b/c their derivatives fall within range of 0 to 0.25 & 0 to 1 resp.

- We can identify it by
 - Weights converging to 0 or stagnation in the improvement of model's performance.
- During training, if loss f' fails to decrease significantly
- If there is erratic behavior in learning curves
- Visualize techniques like Gradient histogram or norms.
- We can solve it by
 - Batch Normalization
 - Activation f' like ReLU (Rectified Linear Unit)
 - Skip connections & Residual Networks
 - LSTM & Gated Recurrent Units (GRUs)
 - Long Short Term Memory Networks
 - Gradient clipping



Exploding is challenge encountered during training deep neural network. It occur when gradients of the network's loss f^n w.r.t weights become excessively large.

Occur when during Backpropagation, derivatives or slopes of neural network's layers grow progressively larger. Root cause lies in weights of the network, rather than choice of Activation f^n .

High weight values lead to correspondingly high derivatives, causing significant deviation in new weight values from previous ones.

$$\Delta W_i = \alpha \cdot \frac{\partial L}{\partial W_i}$$

when gradient become very large, after result is ~~the~~ ^{Greater than 1.}

Identify :-

- loss f^n exhibits erratic behaviour, oscillating wildly instead of steadily decreasing.
- Training process encounter "NaN" (Not a Number) values in loss f^n or other intermediate calculations.
- Tensor board can be used to visualize gradient flowing through network

Solve

- Gradient Clipping
- Batch Normalizes

Dis Advantage of GD :-

1. Choice of Learning Rate
2. Sensitivity to initialization
3. Time Consuming
4. Local Optima

GRADIENT DESCENT VARIATION

1. Batch Gradient descent aka Vanilla Gradient
2. Stochastic Gradient Descent $\rightarrow$ Update weight after each sample
3. Mini Batch Gradient $\rightarrow$ Small batch of sample eg 32, 64

Entire dataset

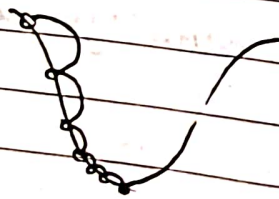
OPTIMIZERS

are algorithms or methods used to minimize an error function (loss f^n) or to maximize the efficiency of production.

Types of Optimizers

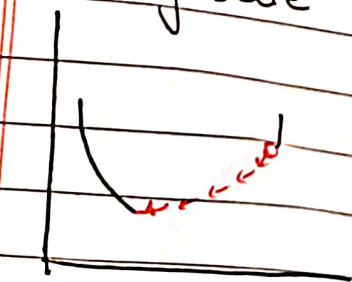
Gradient Descent

Based on convex f^n & iterate its parameters iteratively to minimize a given f^n to its local minimum.

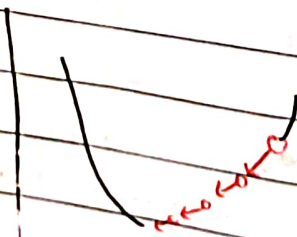


$$W_{new} = W_{old} - \alpha \frac{\partial (Loss)}{\partial (W_{old})}$$

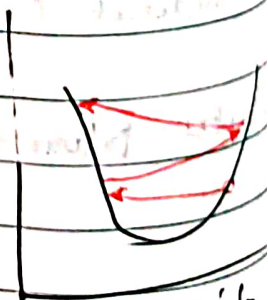
Learning rate



Too low



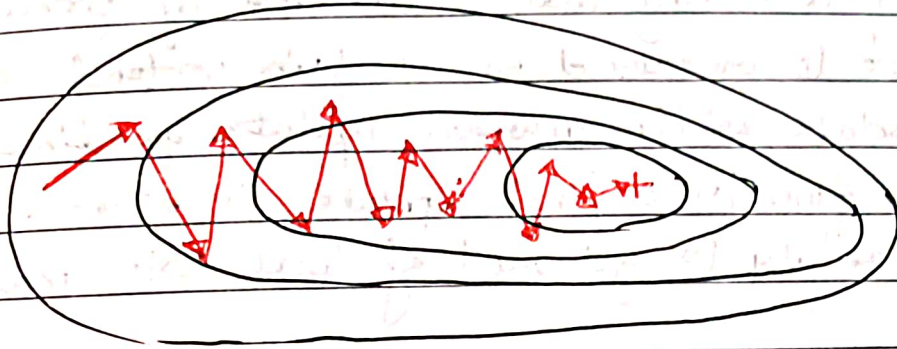
Just Right



Too high

Stochastic Gradient Descent

It update the model parameters one by one. If model has 10k dataset SGD will update the model parameters 10k times,



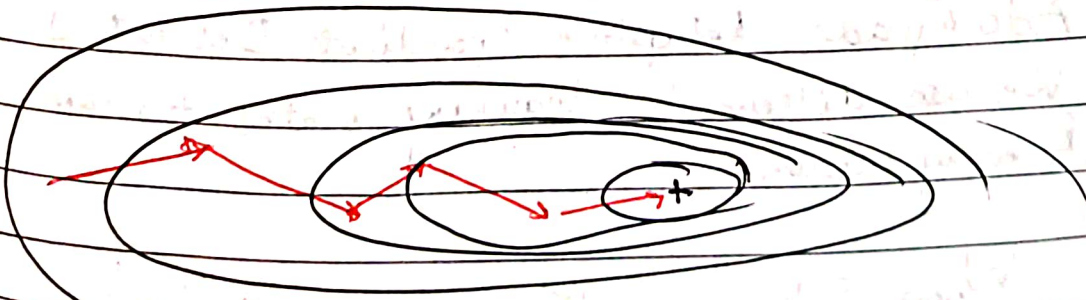
- frequent updates of model Parameter
- Require less Memory
- Allows the use of large data sets as it has to update only one example at a time.

MINI BATCH GRADIENT.

Combination of SGD & batch Gradient. Simply splits the training dataset into small batches & performs an update for each of those batches.

it can reduce the variance when parameters are updated & convergence is more stable.

Split dataset in batches in b/w 50 to 256 chosen at random.

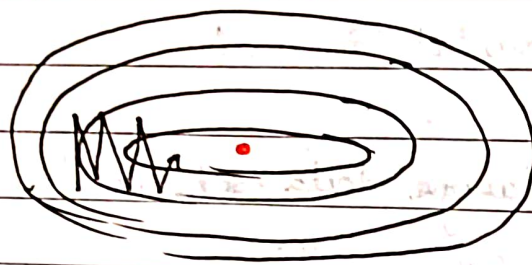


More Stable convergence
more efficient gradient calculation.

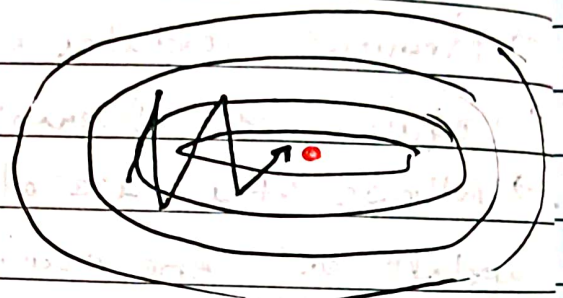
SGD with momentum

adds momentum term to regular SGD.

Momentum stimulates the inertia of an object when it is moving that is direction of previous update is retained to certain extent during the update while current update gradient is used to fine tune final update direction. Have ability to get rid of local optimization.



without Momentum



with Momentum

$$V_{\text{new}} = \eta * V_{\text{old}} - \alpha * \frac{\partial (\text{Loss})}{\partial (W_{\text{old}})}$$

- Momentum help reduce noise
- Exponential weighted avg is used to smoothen the curve.

Ada Grad (Adaptive Gradient Descent)
We use different learning rates for each & every neuron.

$$W_{\text{new}} = W_{\text{old}} + \alpha \frac{\partial (\text{Loss})}{\partial (W_{\text{old}})} \sqrt{\text{cache}_{\text{new}} + \epsilon}$$

- Learning Rate changes adaptively with iteration
- It is able to train sparse data well.

Disadvantage

Learning rate become small no. will cause dead neuron problem

RMS-Prop (Root Mean Square Propagation)

Special version of ~~Adagrad~~ in which learning rate is an exponential avg of gradients. Instead of cumulative sum of square gradients.

basically combines momentum with AdaGrad.

$$\text{cache}_{\text{new}} = \gamma * \text{cache}_{\text{old}} + (1 - \gamma) * \left(\frac{\partial(\text{Loss})}{\partial(w_{\text{old}})} \right)^2$$

Disadvantage
Slow learning

ADAM (Adaptive Moment Estimation)

It is a method that computes adaptive learning rates for each parameter.

It stores both the decaying average of past Gradients, similar to momentum & also the decaying average of ^{past} squared gradients, similar to RMS prop & Adadelta. Combination of two Method.

① Gradient Descent with Momentum

② RMS Propagation.

① Momentum at time t Learning Rate

2. RMS Prop.

Update at t

$$w_{t+1} = w_t - \alpha m_t$$
$$m_t = \beta m_{t-1} + (1 - \beta) \frac{\partial E}{\partial w_t}$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} * \frac{\partial E}{\partial w_t}$$
$$v_t = \beta v_{t-1} + (1 - \beta) \left(\frac{\partial E}{\partial w_t} \right)^2$$

Aggregate of Gradient at time t

Moving Avg Constant
Eq ① + ②

$$w_{t+1} = w_t - \alpha m_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} * \frac{\partial E}{\partial w_t}$$

$$w_{t+1} = w_t - \gamma \left[m_t + \frac{1}{\sqrt{v_t + \epsilon}} * \frac{\partial E}{\partial w_t} \right]$$

Advantage of Adam

Easy to Implement

Computationally Efficient

Little Memory requirements

How To Choose Optimizer

→ If data is sparse use the self applicable methods namely Adagrad, Adadelta, RMSprop, Adam.

Adam just added bias-correction & Momentum on the basis of RMSprop.

As gradient becomes sparse, Adam will perform better than RMSprop.

(Image Classification)	Large Dataset	→	SGD
(Training a deep CNN)	Non-convex Problems	→	Momentum
NLP	Sparse data	→	Adagrad (NLP tasks)
(LSTM)	Non-stationary Data	→	RMSprop (RNN, LSTM)
RiskNet ImageNet	General Purpose	→	Adam (General Purpose)
GAN	Faster Convergence	→	Nadam



ACTIVATION f^n

are mathematical f^n applied to the output of a neuron. It introduces non-linearity into the model, allowing network to learn & represent complex patterns in the data. with non linearity neural network behaves like a linear regression model, no matter how many layers it has.

Non-linearity means that the relationship b/w i/p & o/p is not a straight line.

It means o/p does not change proportionally with i/p.

By adding non-linear activation f^n (like ReLU, Sigmoid, or Tanh) the network can create curve decision boundaries to separate them correctly.

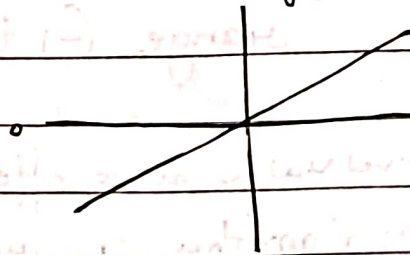
Types :-

1. Linear Activation f^n $y=x$
Range $(-\infty \text{ to } \infty)$

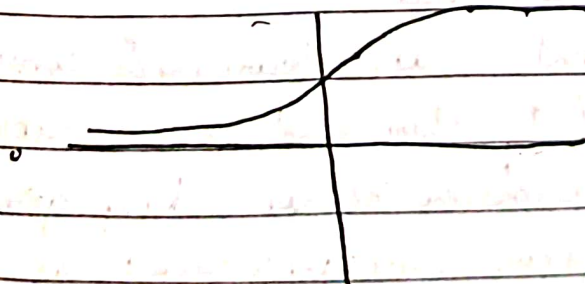
→ is used at just one place i.e o/p layer

Main use case is in o/p layer of a neural network used for regression.

Rarely used in hidden layers.



2. Sigmoid



→ Represented as

$\sigma(x)$ is a smooth, continuously differentiable fⁿ

$$f(x) = \frac{1}{1 + e^{-x}}$$

It takes real value i/o & squashes it to value b/w 0 & 1.

o/p can easily interpreted as probabilities, which makes it natural for binary classification problems.

Sigmoid suffer "vanishing gradient"

as fⁿ saturates at 0 or 1 with an extremely flat slope. In these region gradient is very close to 0.

This result in very small changes in weights during Back Propagation.

TANH (hyperbolic tangent)

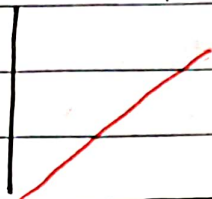
$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

range (-1 to +1)

- It can deal with -ve value more effectively.
- It can help learning algorithm converge faster b/c it has zero-centered.
- Stronger gradient during training, faster learning & converging against the problem of vanishing gradients, become very small (close to zero).

- frequently used in hidden layers.
- It can result in more efficient training.

ReLU (Rectified Linear Unit)



$$f(x) = \max(0, x)$$

It threshold the input at zero, returning 0 for -ve values

- this property is critical in mitigating the vanishing gradient problem.

Technically ReLU is non linear fⁿ b/c it has non-differentiable point at $x=0$.

- It naturally leads to sparse activation; more efficient comput.
- Computationally inexpensive.

Softmax



for i/p vector x with element $x_1, x_2, \dots, x_c$:

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Normalized exponential function

- useful within the context of multiclass classification Problem.
- operates on vector (logits) which represents the raw predictions or scores for each class computed by previous layers of neural network.

Q. O/P of Softmax is probability distribution that sum up to one. Each element of o/p represent probability that the i/p belong to particular class.

CHOOSING RIGHT ACTIVATION f^n

For Binary Classification Sigmoid f^n

Multiclass classification Softmax f^n

If unsure use ReLU for hidden layers.

Most common default A/f^n & usually a good choice.

ERROR f^n

mathematical f^n that measures how well a model prediction match the true outcomes.

The goal is to guide optimization algorithms.

IMPORTANCE :-

- Guide Model Training
- Measure Performance
- Influence learning dynamics

Types of Loss f^n

1. Regression Loss f^n

(a) MSE (Mean Squared Error) Loss

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

(b) MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

(c) Huber loss

combines advantages of MSE & MAE, less sensitive to outliers than MSE & differentiable everywhere unlike MAE.

defined as:

$$\begin{cases} \frac{1}{2} (y_i - \hat{y}_i)^2 & \text{for } |y_i - \hat{y}_i| \leq \delta \\ \delta |y_i - \hat{y}_i| - \frac{1}{2} \delta^2 & \text{for } |y_i - \hat{y}_i| > \delta \end{cases}$$

↖ Tuning Parameters

(d) Log-Cosh Loss

Smooth loss f'' for regression defined as logarithm of hyperbolic cosine of prediction error.

$$\text{Log-Cosh} = \sum_{i=1}^n \log(\cosh(\hat{y}_i - y_i))$$

$$\cosh = \frac{e^x + e^{-x}}{2}$$

Classification Loss

(a) Binary Cross-Entropy Loss (Log loss)

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

disadv. can be sensitive to imbalanced dataset.

(b) Categorical Cross Entropy Loss

used for Multiclass classification

$$-\sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij})$$

disadv. Not suitable for Sparse target

(c) Sparse Categorical cross entropy

$$= -\sum_{i=1}^n \log(\hat{y}_{i, y_i})$$

(d) Kullback-Leibler Divergence Loss (KL Divergence)

how one probability distribution diverges from second, expected probability distribution.

$$\text{KL Divergence} = \sum_{k=1}^n \sum_{j=1}^k y_{kj} \log\left(\frac{y_{kj}}{\hat{y}_{kj}}\right)$$

(e) Hinge Loss *used especially for SVM*

$$= \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \cdot \hat{y}_i)$$

(f) Squared Hinge Loss

$$= \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \cdot \hat{y}_i)^2$$

(g) Focal loss = $-\frac{1}{n} \sum_{i=1}^n (1 - \hat{y}_i)^{\gamma} \log(\hat{y}_i)$ $\gamma \rightarrow$ focusing Parameter

3. Ranking Loss f^n

a) Contrastive Loss

b) Triplet Loss

c) Margin ranking Loss

4. Image & Reconstruction Loss f^n

a) Pixel-wise Cross Entropy Loss

b) Dice Loss

c) Jaccard Loss (Intersection over Union)

d) Perceptual Loss

e) Total Variation Loss

5) Adversarial Loss f^n

a) Adversarial Loss (GAN Loss)

b) Least Squares GAN Loss

6) Specialized Loss f^n

A) CTC Loss (Connectionist Temporal Classification)

B) Poisson Loss

C) Cosine Proximity Loss

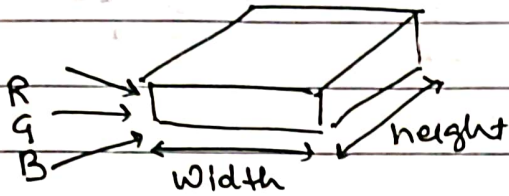
D) Log Loss

E) Earth Mover's Distance (Wasserstein Loss)

RNN

CNN special type of feed forward ANN in which connectivity pattern b/w its neuron is inspired by ~~the~~ visual cortex. CNN are also known as convnets.

Here dimensions of image are represented by RGB

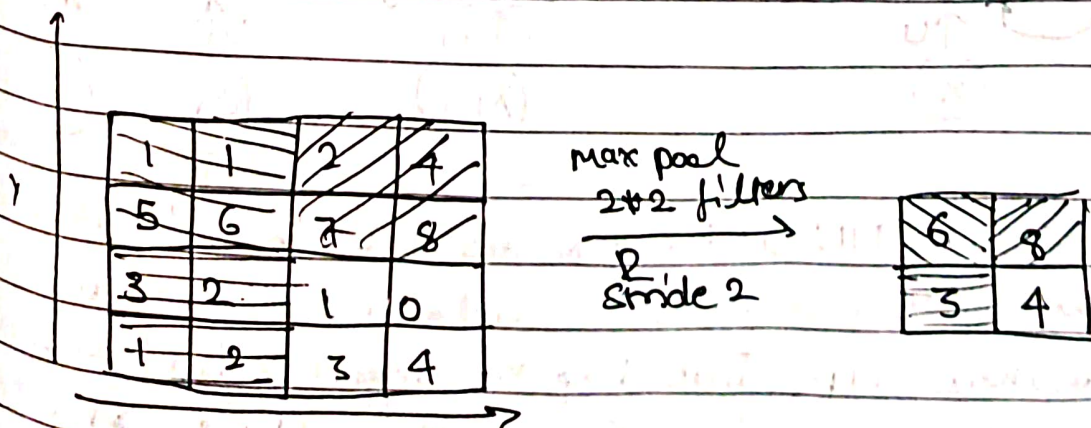


Input: Image consists of $[32 \times 32 \times 3]$
width height $\rightarrow$ Colour channel

Convolutions: Dot product b/w weights & small region which they are actually linked to in the input volume.
for: we choose to incorporate 12 filters, then it will result in volume of $[32 \times 32 \times 12]$

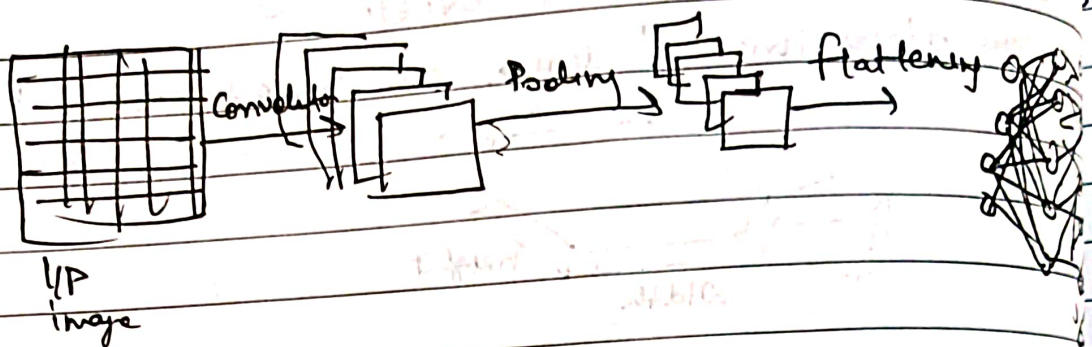
ReLU - $\max(0, x)$ thresholding at zero
It results in $[32 \times 32 \times 12]$ ~~unchanged~~ unchanged size of volume.

Pooling $\rightarrow$ this is used to perform downsampling operation along the spatial dimension (Width & height) that result in $[16 \times 16 \times 12]$ volume



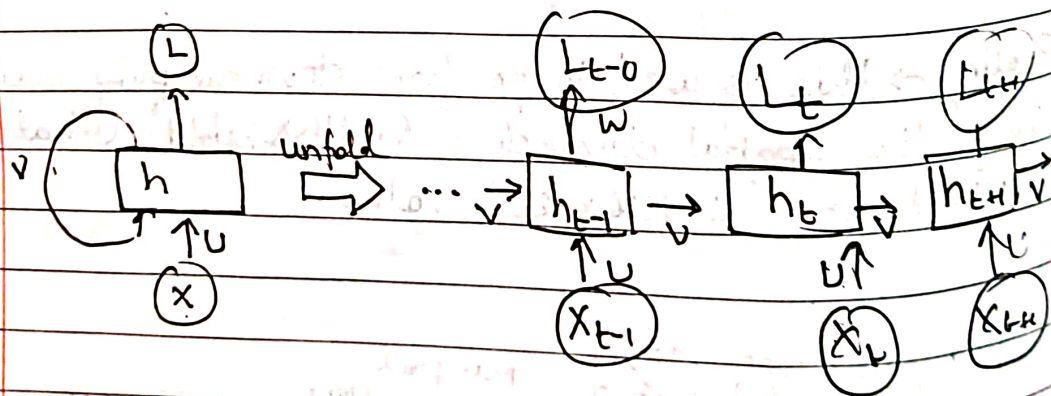
ANN takes a lot of time to reach optimum
So we use CNN

Locally connected



RNN

RNN allow the network to remember past info by feeding the o/p from one step into next step. This help the network understand the context of what has already happened & make prediction better.



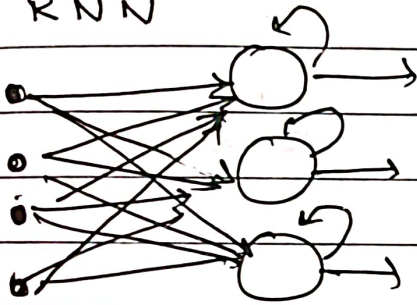
feed forward NN (FNN) process data in 1 direction from I/P to o/p without retaining info from previous i/p. This help in image classification but struggle with Sequential data since they lack memory



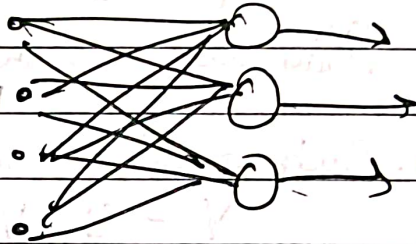
- CNN can handle transition in object position
- Reduce no of i/p neuron string

RNN solve by incorporating loops that allows info from previous step to feed back into the network.

a) RNN

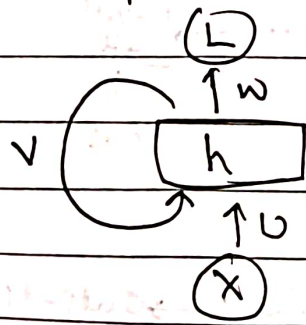


(b) feed forward



Key COMPONENT OF RNN :-

Recurrent Neurons - fundamental processing unit in RNN is Recurrent Unit. can "remember" info from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



RNN Unfolding is process of expanding recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in series illustration.

Unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the networks weights enhancing the RNN's ability to learn dependencies within sequential data.

ARCHITECTURE

RNN use shared weights across time steps, allowing them to remember info over sequence

In RNN Hidden state H_i is calculated for every i p X_i to retain sequential dependence.

1. Hidden State Calculation

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Annotations:
- h : Current Hidden state
- U : Weight Matrices
- W : Weight Matrices
- B : Bias

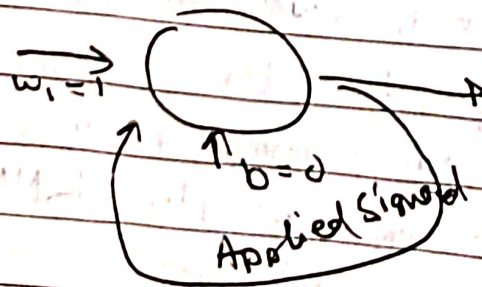
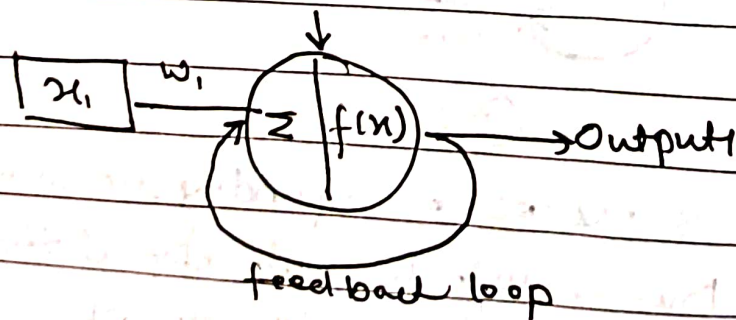
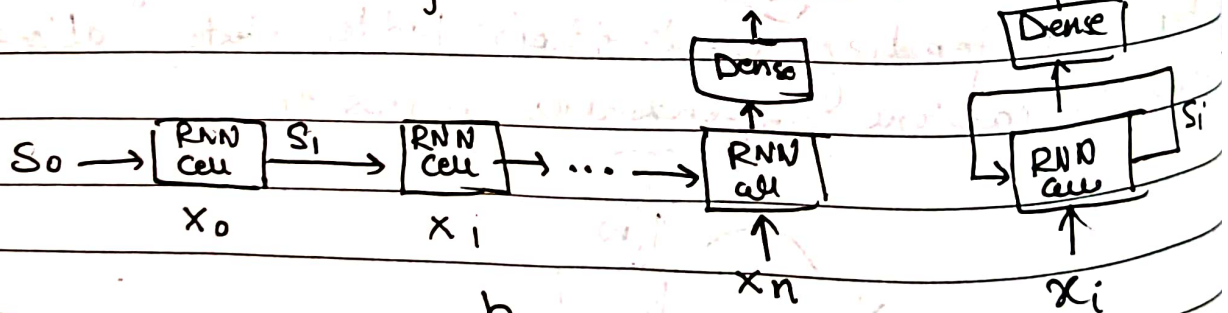
2. Output Calculation:

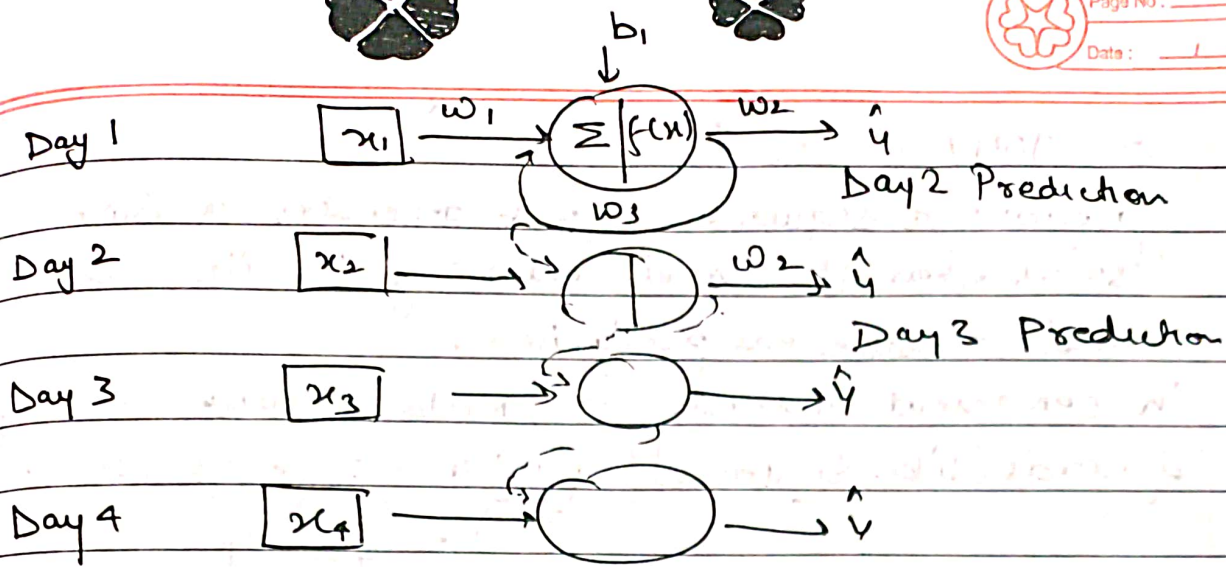
$$Y = O(V \cdot h + C)$$

Annotations:
- O : Activation f^h
- $V \cdot h + C$: weights & bias

3. Overall f^h

$$Y = f(X, h, W, U, V, B, C)$$





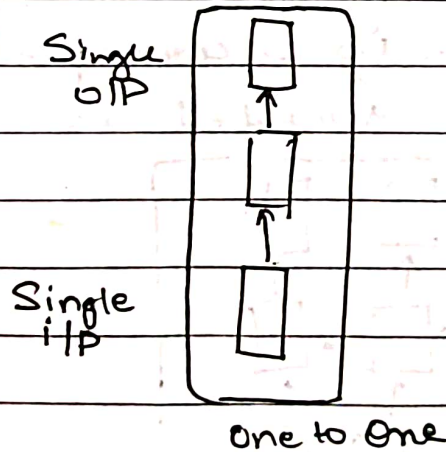
TYPES OF RNN :-

based on number of i/p & o/p in the network.

1. One to One RNN

Simplest type of neural network architecture where there is a single i/p & a single o/p.

Straightforward classification tasks such as Binary Classification where no sequential data is involved.

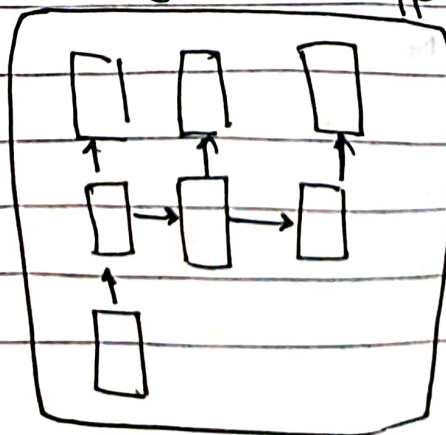


2. One-to-Many

Single i/p produces Multiple o/p over time. Ex: in image captioning a single Image can be used as i/p to generate a seq' of words as caption

Multiple o/p's

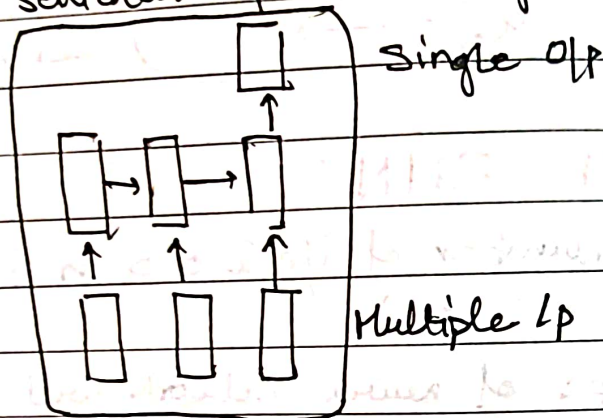
Single i/p



3. MANY TO ONE RNN

Receives a sequence of i/p & generates a single o/p.
Useful when the overall context of the i/p sequence is needed to make one prediction.

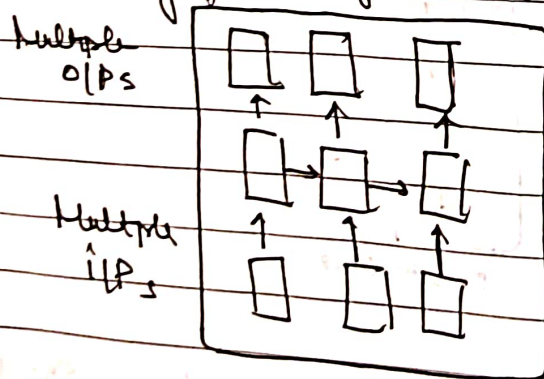
→ In sentiment analysis the model receives a seq. of words (like sentence) & produces a single o/p like +ve, -ve.



4. MANY TO MANY

processes a sequence of i/p & generates a seq. of o/p.

In language translation task a seq. of words in one lang. is given as i/p & corresponding seq. in another language is generated as o/p.



Variants of Recurrent Neural Networks :-
1. Vanilla RNN

- consists of single hidden layer where weights are shared across time steps. Suitable for learning short term dependencies but are limited by vanishing gradient problem.

3. Bidirectional RNN

Process i/p in both forward & Backward directions capturing both Past & future context for each time step.

Entity Recognition & Question answering.

3. Long Short-Term Memory Networks (LSTMs)

introduce memory mechanism to overcome vanishing gradient problem.

Each LSTM cell has three gates :-

Input Gate

Forget Gate

O/P Gate

4. Gated Recurrent Units (GRUs)

Simplify LSTM by combining i/p & forget gates into single update gate & streamlining the o/p mechanism.